E-COMMERCE BACKEN

Developed using Spring Boot, MySQL, and REST API architecture.

1. PROJECT OVERVIEW

This project is a backend system for an e-commerce application that enables users to register, login, manage products, handle cart operations, and place orders.

The system is designed using RESTful architecture and follows industry best practices.

Objectives

- Build scalable backend APIs
- Implement authentication using tokens
- Manage cart and order system
- Provide API documentation using Swagger

2. TECHNOLOGIES USED

Java 17 – Programming Language

Spring Boot – Backend Framework

Spring Data JPA – ORM

MySQL – Database

Swagger – API Documentation

Maven – Build Tool

Lombok – Boilerplate Reduction

3. SYSTEM ARCHITECTURE

The application follows a layered architecture:

Controller Layer → Handles HTTP Requests

Service Layer → Business Logic

Repository Layer → Database Interaction

Database → MySQL

Architecture Flow

Client → Controller → Service → Repository → Database

4. DATABASE DESIGN (ER CONCEPT)

Entities used in the system:

- Seller
- Customer
- Product
- Cart
- CartItem
- Orders
- UserSession

Relationships

- Seller → Product (One-to-Many)
- Customer → Cart (One-to-One)
- Cart → CartItem (One-to-Many)
- CartItem → Product (Many-to-One)
- Customer → Orders (One-to-Many)

5. AUTHENTICATION SYSTEM

The application uses token-based authentication.

Steps:

1. User logs in with mobile & password
2. Server generates a unique token
3. Token is stored in UserSession table
4. Token is required for all secured APIs

6. API DOCUMENTATION

Seller APIs

POST /addseller → Register seller

POST /login → Login seller

GET /seller → Get seller details

PUT /seller → Update seller

Cart APIs

POST /cart/add → Add product to cart

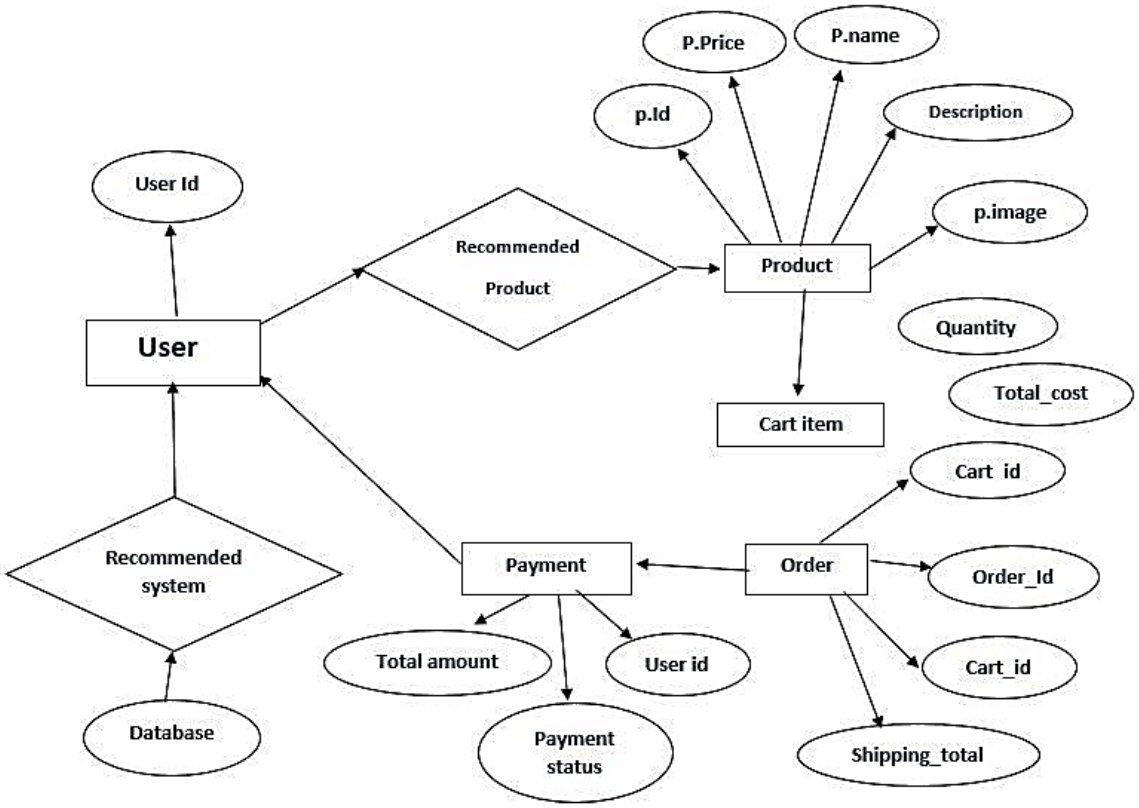
GET /cart → View cart

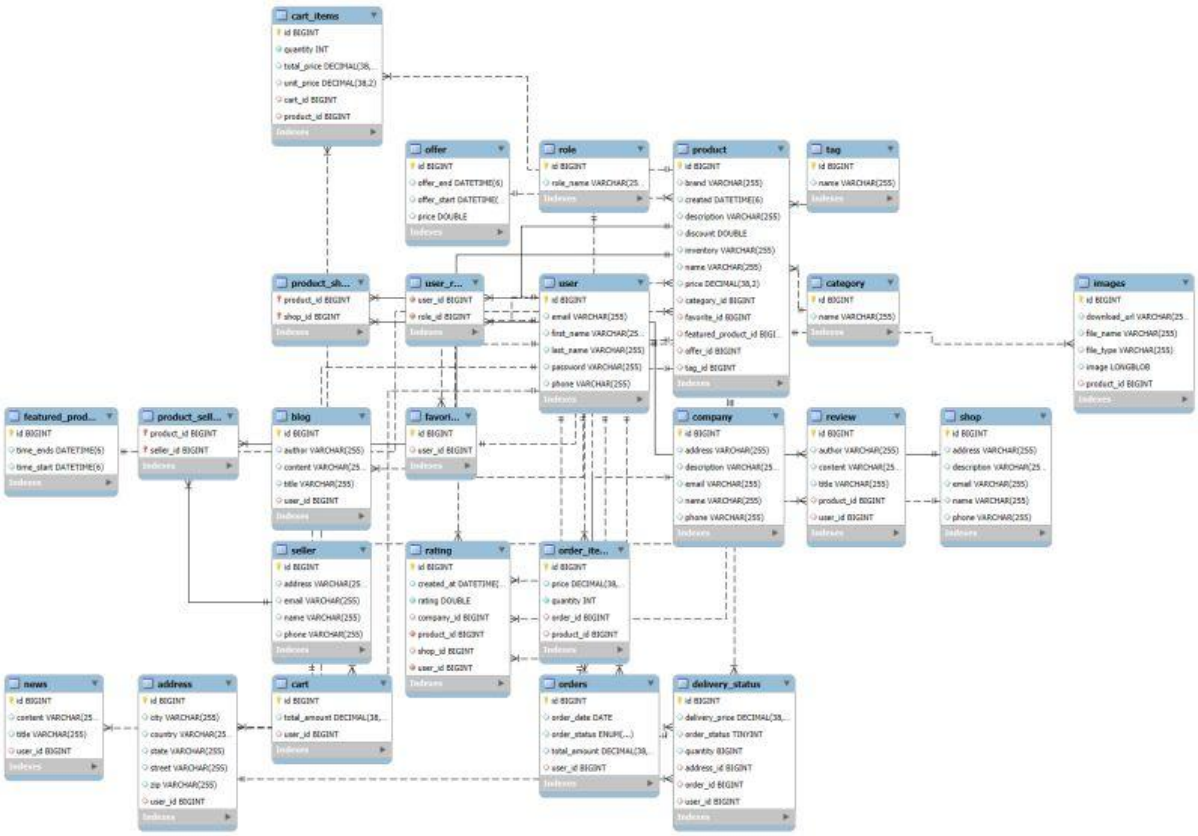
DELETE /cart/remove → Remove item

Example JSON

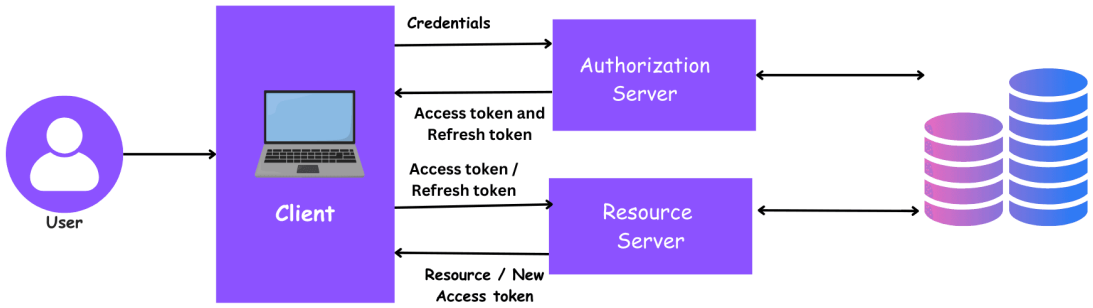
{
"name": "Sravan",
"mobile": "9999999999",
"email": "sravan@gmail.com",
"password": "1234"
}

7. ER DIAGRAM (Database Design)





8. FLOW DIAGRAM (System Flow)



The system follows a **secure token-based workflow** for handling user actions.
Each step ensures authentication and proper data handling within the application.

1. User registers

- The user creates an account by providing required details (name, email, password).
- Data is stored in the database.

2. User logs in and gets token

- The user logs in using credentials.
- The system validates the user.
- A **unique session token** is generated and returned.

3. User sends token in headers

- The user sends the token in the request header:

```
Authorization: Bearer <token>
```

- The backend validates the token before processing any request.

4. User adds items to cart

- The authenticated user selects products.
- Products are added to the cart.
- Cart data is stored in `Cart` and `CartItem` tables.

5. User places order

- The user confirms the cart.
- The system:
  - Calculates total price
  - Creates an order
- Order details are stored in the Orders table.

9. TESTING

Testing performed using Swagger and Postman.

Test Cases:

• Valid login → Token generated

• Invalid login → Error

• Add to cart → Success

• Invalid token → Access denied

10. CONCLUSION

This project demonstrates backend development skills using Spring Boot.

It includes authentication, database design, API development, and real-world e-commerce features.